TRƯỜNG ĐẠI HỌC SÀI GÒN

MÔN HỌC: HỌC SÂU

BÀI TẬP LỚN 01

XẤP XỈ HÀM GAUSSIAN 2D BẰNG MẠNG NƠ-RON NHIỀU TẦNG (MLP) TRÊN PYTORCH

Sinh viên thực hiện: Huỳnh Kiện Khải
Mã số sinh viên: 3123411137

TP. Hồ Chí Minh, tháng 10 năm 2026

1. Giới thiệu

1.1. Mục tiêu

Báo cáo này khảo sát khả năng xấp xỉ hàm số của mạng nơ-ron nhiều tầng (MLP). Hàm mục tiêu là một hàm Gaussian hai biến. MLP được cài đặt bằng PyTorch và được huấn luyện để dự đoán giá trị hàm từ (x_1, x_2). Ba thí nghiệm được thực hiện:

- **Thí nghiệm A:** thay đổi số mẫu huấn luyện, với mạng 1 tầng ẩn, 3 nơ-ron.
- **Thí nghiệm B:** thay đổi số nơ-ron của một tầng ẩn, từ 1 đến 10.
- **Thí nghiệm C:** thay đổi số tầng ẩn, từ 1 đến 4, mỗi tầng 3 nơ-ron.

1.2. Phát biểu bài toán

Hàm cần xấp xỉ là mật độ Gaussian hai chiều:

$$f(x) = 1 / (2\pi \sqrt{|C|}) \cdot \exp(-\tfrac{1}{2} (x - \mu)^T C^{-1} (x - \mu)) \quad (1)$$

với $\mu = [0, 0]^T$ và ma trận hiệp phương sai $C = [[0.3, -0.5], [-0.5, 2]]$. Đầu vào lấy mẫu đều trong miền $x_1 \in [-2, 2]$, $x_2 \in [-4, 4]$. Bài toán là hồi quy: tìm mạng $f_\theta: \mathbb{R}^2 \to \mathbb{R}$ sao cho sai số bình phương trung bình giữa $f_\theta(x)$ và $f(x)$ nhỏ nhất trên dữ liệu chưa thấy.

1.3. Nội dung thực hiện

- Sinh dữ liệu Gaussian, chia train/validation/test với seed cố định và vẽ biểu đồ phân bố.
- Cài đặt MLP, vòng huấn luyện có lưu checkpoint theo validation MSE nhỏ nhất và đánh giá trên tập test bằng PyTorch.
- Chạy ba thí nghiệm A, B, C và so sánh bằng MSE, MAE, RMSE, learning curve và thời gian huấn luyện.
- Chọn mô hình có sai số nhỏ nhất trong ba thí nghiệm A, B, C theo validation MSE, kiểm tra lại và minh họa bề mặt xấp xỉ.

2. Dữ liệu

2.1. Công thức và tham số

Nhãn của mỗi mẫu được tính trực tiếp từ công thức (1) với μ và C ở mục 1.2, không thêm nhiễu. Trong hàm Gaussian_2D (src/data_gen.py), với mỗi điểm x ta tính $k = x - \mu$ rồi $y = \text{hệ_số} \cdot \exp(-\tfrac{1}{2} k^T C^{-1} k)$, với $\text{hệ_số} = 1/(2\pi\sqrt{|C|})$.

2.2. Cách lấy mẫu

Với seed 42, mỗi điểm được lấy mẫu độc lập bằng hàm gen_data_set: $x_1 \sim U(-2, 2)$ và $x_2 \sim U(-4, 4)$. Có hai bộ dữ liệu, giống notebook:

- **Bộ cho mô hình chuẩn:** 1000 mẫu, lưu vào data/DataSet.csv với ba cột x1, x2, y_real.

- **Bộ cho thí nghiệm A, B, C:** 3000 mẫu, chuyển sang float32 và xáo trộn bằng numpy.random.default_rng(42). Thí nghiệm B và C lấy 1000 mẫu đầu tiên. Thí nghiệm A dùng chung tập validation và test với B, C (chia từ 1000 mẫu đầu tiên), còn tập train lấy n mẫu đầu tiên của phần còn lại (từ mẫu thứ 1000 trở đi) với n thay đổi.

2.3. Chia dữ liệu

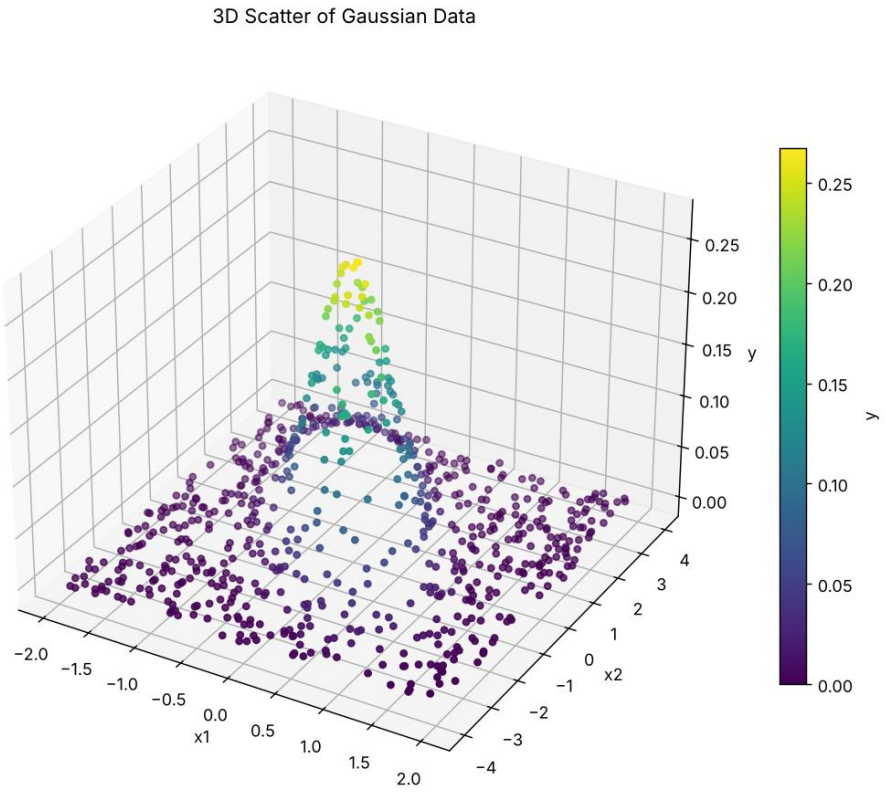
Với 1000 mẫu đầu tiên, hàm data_split_and_to_tensor chia dữ liệu hai lần bằng train_test_split với random_state = 42: 10% làm test, trong 90% còn lại lấy 20% làm validation, phần còn lại là train (720/180/100). Cách chia này dùng cho mô hình chuẩn, B, C và cho tập validation (180 mẫu) và test (100 mẫu) cố định của A. Tập train của A lấy từ phần còn lại của bộ 3000 mẫu (hàm data_A) nên không trùng validation và test; các tập train của A lồng nhau (A nhỏ là tập con của A lớn). Tập validation dùng để chọn checkpoint và chọn kiến trúc; tập test dùng để báo cáo kết quả. Kích thước các tập được cho ở Bảng 1.

Bảng 1. *Kích thước các tập dữ liệu.*

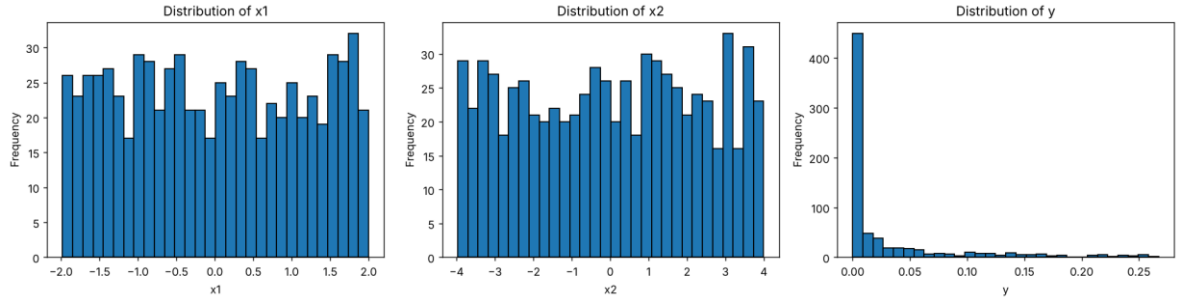
Thí nghiệm	Tổng n	Train	Validation	Test
Mô hình chuẩn, B, C	1000	720	180	100
A	480	200	180	100
A	680	400	180	100
A	880	600	180	100
A	1080	800	180	100
A	1280	1000	180	100
A	1480	1200	180	100
A	1680	1400	180	100
A	1880	1600	180	100
A	2080	1800	180	100
A	2280	2000	180	100

2.4. Biểu đồ dữ liệu

Hình 1 là tập train (720 mẫu) của mô hình chuẩn trong không gian (x_1, x_2, y). Hình 2 là phân bố của x_1, x_2 và y. Hai biến đầu vào phân bố gần đều trong miền lấy mẫu, đúng như kỳ vọng từ phân phối đều. Phân bố của y lệch mạnh về 0: phần lớn mẫu nằm ở vùng "đuôi" phẳng của hàm Gaussian, chỉ một số ít mẫu nằm gần đỉnh.



Hình 1. *Dữ liệu train trong không gian (x1, x2, y); màu thể hiện giá trị y.*



Hình 2. *Phân bố của x1, x2 và y trên tập train.*

3. Mô hình và cài đặt

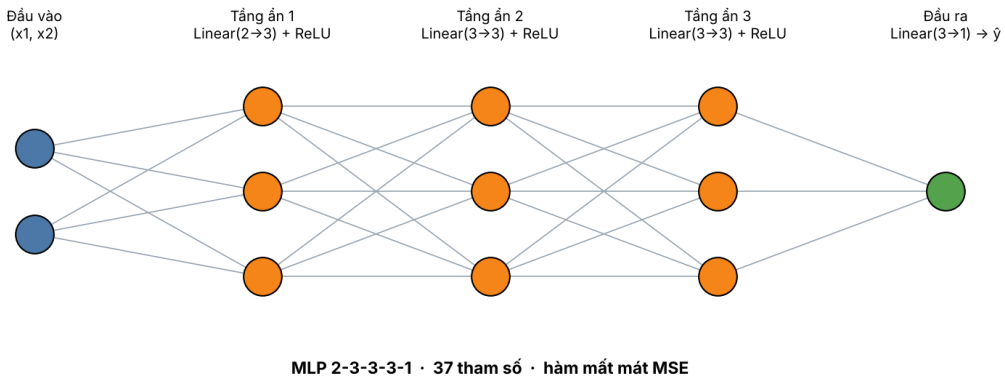
3.1. Kiến trúc MLP

Mạng gồm L tầng ẩn, mỗi tầng là một phép biến đổi tuyến tính theo sau là hàm kích hoạt ReLU, và tầng đầu ra là một phép tuyến tính, không có kích hoạt vì đây là bài toán hồi quy. Với $h^{(0)} = x \in \mathbb{R}^2$, lan truyền thuận (forward) được viết như sau:

$$h^{(l)} = ReLU(W^{(l)} h^{(l-1)} + b^{(l)}),\ l = 1, ..., L \quad (2)$$

$$\hat{y} = W^{(L+1)} h^{(L)} + b^{(L+1)} \quad (3)$$

Hình 3 là sơ đồ kiến trúc 2-3-3-3-1 của mô hình được chọn ở mục 4.6 (3 tầng ẩn, mỗi tầng 3 nơ-ron), kèm kích thước từng lớp và tổng số tham số. Các thí nghiệm dùng chung một lớp MLP: thí nghiệm B đổi số nơ-ron của một tầng ẩn, thí nghiệm C đổi số tầng ẩn.



Hình 3. *Sơ đồ kiến trúc của mô hình được chọn (2-3-3-3-1).*

3.2. Hàm mất mát và các độ đo

Hàm mất mát khi huấn luyện là sai số bình phương trung bình (MSE) trên N mẫu. Khi đánh giá ta báo cáo thêm RMSE và MAE:

$$MSE = (1/N)\ \Sigma\ (\hat{y}_i - y_i)^2,\ \ RMSE = \sqrt{MSE},\ \ MAE = (1/N)\ \Sigma\ |\hat{y}_i - y_i| \qquad (4)$$

3.3. Thiết lập huấn luyện

Mọi thí nghiệm dùng chung một thiết lập (Bảng 2). Trước mỗi lần huấn luyện, ta gọi torch.manual_seed(42) để khởi tạo trọng số và thứ tự xáo trộn batch giống nhau giữa các lần chạy lại.

Bảng 2. *Siêu tham số và thiết lập chung.*

Thành phần	Giá trị
Seed	42
Bộ tối ưu	Adam, learning rate = 0.002
Hàm kích hoạt	ReLU (tầng ẩn); tầng ra tuyến tính
Hàm mất mát	MSE
Số epoch	1000
Batch size	32 (mô hình chuẩn dùng full-batch 720 mẫu)
Chọn checkpoint	Epoch có validation MSE nhỏ nhất
Thiết bị	CPU
Thí nghiệm A	1 tầng ẩn × 3 nơ-ron; số mẫu train 200 ... 2000
Thí nghiệm B	n = 1000; 1 tầng ẩn, B = 1 ... 10 nơ-ron
Thí nghiệm C	n = 1000; C = 1 ... 4 tầng ẩn, mỗi tầng 3 nơ-ron

3.4. Các đoạn mã quan trọng

Lớp MLP (file src/model.py) xây mạng từ danh sách số nơ-ron của các tầng ẩn hiden_sizes. Mỗi phần tử của hiden_sizes tạo thêm một cặp Linear và ReLU; cuối cùng thêm tầng Linear đầu ra.

```
class MLP(nn.Module):
  def __init__(self, input_size = 2, hiden_sizes = [3], output_size = 1):
    super().__init__()
    layers = []
    prev_input = input_size

    for hiden_size in hiden_sizes:# tạo ra len(hiden_sizes) hiden layers
      layers.append(nn.Linear(prev_input, hiden_size))
      layers.append(nn.ReLU()) # hàm kích hoạt ReLU
      prev_input = hiden_size

    layers.append(nn.Linear(prev_input, output_size))
    self.network = nn.Sequential(*layers)

  def forward(self, x):
      return self.network(x)
```

Hàm Train (file src/train.py) gồm hai pha ở mỗi epoch. Ở pha train, với mỗi batch ta xóa gradient, tính mất mát, lan truyền ngược và cập nhật trọng số. Ở pha validation, mô hình chuyển sang eval() và tính MSE trên toàn bộ tập validation trong torch.no_grad(), tức là validation không cập nhật trọng số. Checkpoint chỉ được ghi khi validation MSE tốt hơn mọi epoch trước.

```
    for X_batch, y_batch in train_loader:
      X_batch, y_batch = X_batch.to(device), y_batch.to(device)
      optimizer.zero_grad() # reset grad về 0 để không bị cộng dồn
      y_pred = model(X_batch).squeeze(-1) # train + predict
      loss = criterion(y_pred, y_batch) # tính loss
      loss.backward() # backpropagation
      optimizer.step()
      running_loss += loss.item() * X_batch.size(0) # cộng dồn loss

    # validation
    model.eval()
    with torch.no_grad():
      y_val_pred = model(X_val_tensor).squeeze(-1)
      val_loss = criterion(y_val_pred, y_val_tensor)

    # lưu checkpoint min_vali_loss
    if val_loss.item() < min_vali_loss:
      min_vali_loss = val_loss.item()
      best_epoch = epoch + 1
      torch.save({"epoch": epoch + 1, "model_state_dict": model.state_dict(),
                  "optimizer_state_dict": optimizer.state_dict(),
                  "val_loss": min_vali_loss}, file)
```

Sau khi huấn luyện, hàm evaluate (file src/evaluate.py) nạp lại checkpoint tốt nhất và tính MSE, RMSE, MAE trên tập test. Như vậy số liệu test ứng với epoch được chọn bằng validation, không phải epoch cuối.

```
def evaluate(model, file, X_test, y_test):
    checkpoint = torch.load(file, map_location=device, weights_only=True)
    model.load_state_dict(checkpoint["model_state_dict"])
    model.eval()
    with torch.no_grad():
        pred = model(X_test.to(device)).squeeze(-1).cpu()

    err = pred - y_test
    mse = (err ** 2).mean().item()
    return {"test_mse": mse, "test_rmse": mse ** 0.5, "test_mae":
err.abs().mean().item()}
```

3.5. Môi trường chạy

Python 3.13, PyTorch 2.14 (chạy trên CPU), NumPy 2.5, pandas 3.0, scikit-learn 1.9, Matplotlib 3.11. Danh sách phụ thuộc nằm trong requirements.txt; các đường dẫn trong mã đều tương đối so với thư mục gốc dự án.

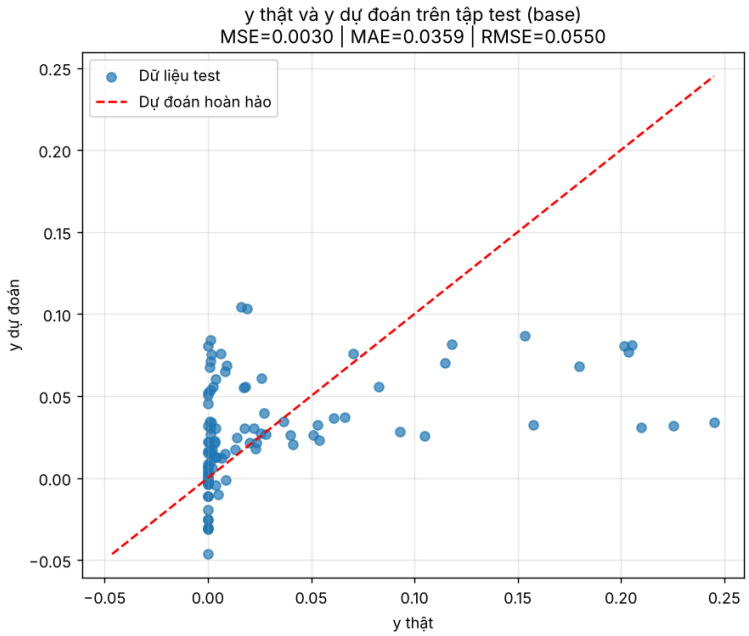
4. Thực nghiệm

4.1. Quy trình đánh giá

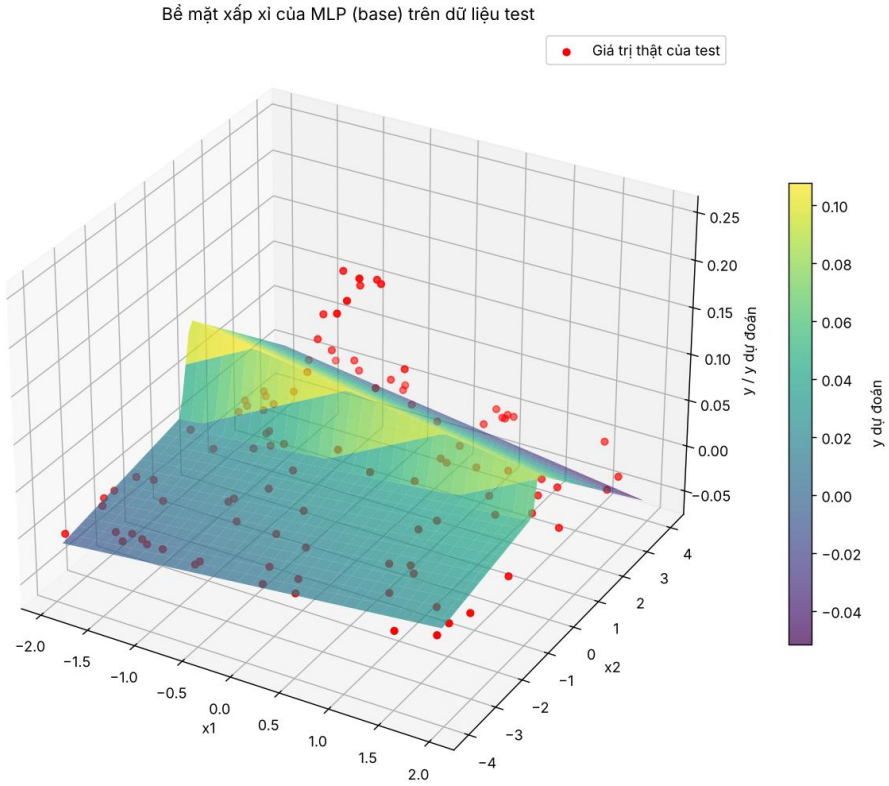
Các cấu hình trong cùng một thí nghiệm chỉ khác nhau đúng yếu tố đang khảo sát: cùng seed, bộ tối ưu, learning rate, batch size, số epoch và cách chia dữ liệu. Checkpoint của mỗi cấu hình được chọn theo validation MSE, và test được tính cho checkpoint đó. Ở cả A, B và C, mọi cấu hình dùng cùng tập validation (180 mẫu) và cùng tập test (100 mẫu); ở A chỉ có số mẫu train thay đổi (200 đến 2000), nên các điểm của A so sánh trực tiếp được với nhau và với B, C.

4.2. Mô hình chuẩn

Mô hình chuẩn là mạng 1 tầng ẩn × 3 nơ-ron, huấn luyện full-batch trên 720 mẫu. Kết quả trên test: MSE = 3.03×10^{-3}, RMSE = 0.0550, MAE = 0.0359; epoch tốt nhất là 1000. Hình 4 so sánh y dự đoán với y thật trên tập test và Hình 5 là bề mặt xấp xỉ của mô hình cùng các điểm test. Mạng 3 nơ-ron chưa bắt được đỉnh của hàm: các mẫu có y lớn bị dự đoán thấp hơn nhiều (Hình 4) và bề mặt chỉ gồm vài mảnh phẳng (Hình 5).



Hình 4. *y thật và y dự đoán trên tập test của mô hình chuẩn.*



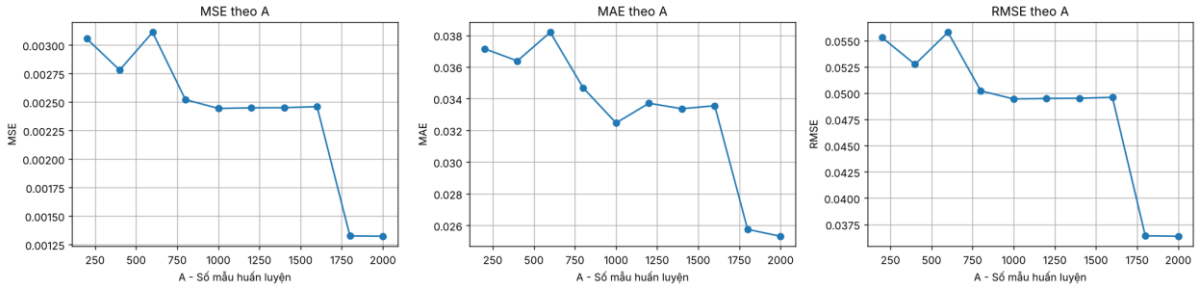
Hình 5. *Bề mặt xấp xỉ của mô hình chuẩn và các điểm test.*

4.3. Thí nghiệm A: số mẫu huấn luyện

Mạng 1 tầng ẩn × 3 nơ-ron được huấn luyện với số mẫu train từ 200 đến 2000, validation (180 mẫu) và test (100 mẫu) cố định (Bảng 3, Hình 6).

Bảng 3. *Kết quả thí nghiệm A (đo trên test).*

A (số mẫu train)	MSE	MAE	RMSE
200	3.06×10^{-3}	0.0371	0.0553
400	2.78×10^{-3}	0.0364	0.0527
600	3.11×10^{-3}	0.0382	0.0558
800	2.52×10^{-3}	0.0347	0.0502
1000	2.44×10^{-3}	0.0325	0.0494
1200	2.45×10^{-3}	0.0337	0.0495
1400	2.45×10^{-3}	0.0334	0.0495
1600	2.46×10^{-3}	0.0336	0.0496
1800	1.33×10^{-3}	0.0258	0.0364
2000	1.32×10^{-3}	0.0253	0.0364



Hình 6. *MSE, MAE và RMSE trên test theo số mẫu train A.*

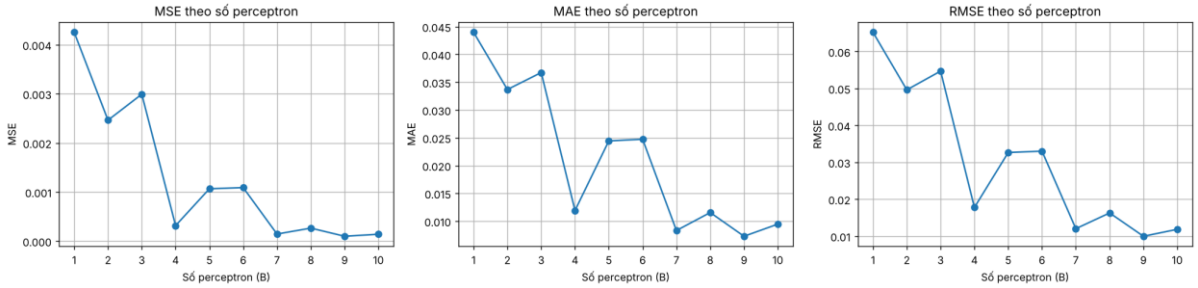
MSE test giảm từ 3.06×10^{-3} (A = 200) xuống 1.32×10^{-3} (A = 2000), cũng là giá trị thấp nhất. Đường cong gần như phẳng (2.4×10^{-3} đến 2.5×10^{-3}) từ A = 800 đến A = 1600, rồi giảm gần một nửa ở A = 1800; ở phần đầu, A = 600 (3.11×10^{-3}) cao hơn A = 400 (2.78×10^{-3}). MAE và RMSE có xu hướng tương tự MSE.

4.4. Thí nghiệm B: số nơ-ron của tầng ẩn

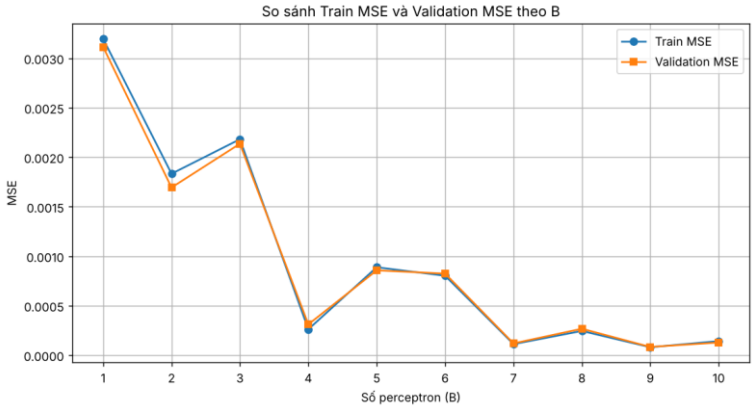
Với 1000 mẫu và 1 tầng ẩn, số nơ-ron B tăng từ 1 đến 10 (Bảng 4, Hình 7, 8, 9).

Bảng 4. *Kết quả thí nghiệm B (1 tầng ẩn, n = 1000).*

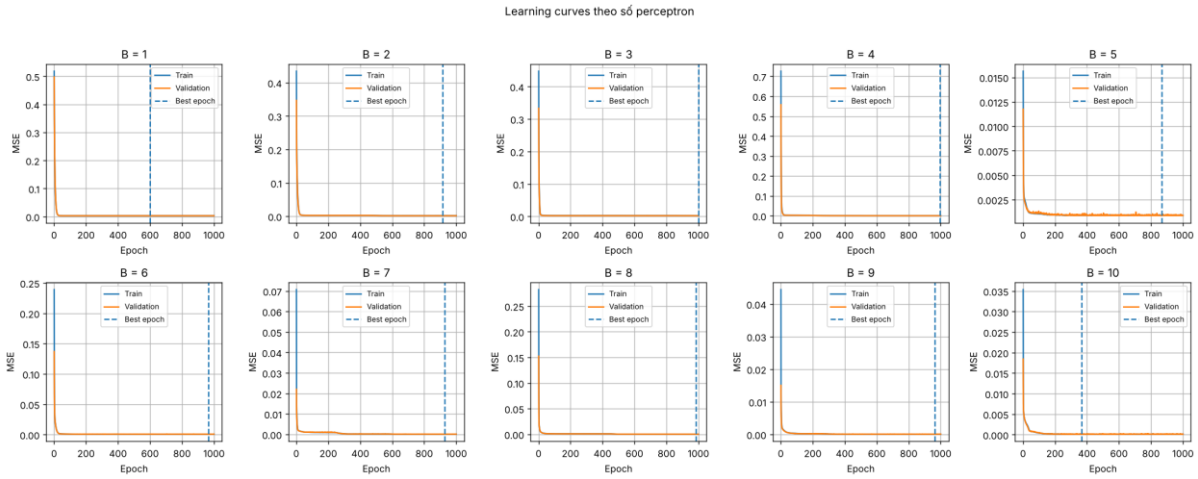
B	Epoch tốt nhất	Train MSE	Val MSE	Test MSE	Test MAE	Test RMSE
1	599	3.20×10^{-3}	3.11×10^{-3}	4.25×10^{-3}	0.0440	0.0652
2	914	1.83×10^{-3}	1.69×10^{-3}	2.46×10^{-3}	0.0337	0.0496
3	999	2.18×10^{-3}	2.14×10^{-3}	2.99×10^{-3}	0.0367	0.0547
4	995	2.61×10^{-4}	3.11×10^{-4}	3.16×10^{-4}	0.0119	0.0178
5	866	8.88×10^{-4}	8.57×10^{-4}	1.07×10^{-3}	0.0244	0.0327
6	966	8.02×10^{-4}	8.24×10^{-4}	1.09×10^{-3}	0.0247	0.0330
7	929	1.12×10^{-4}	1.19×10^{-4}	1.45×10^{-4}	0.0083	0.0120
8	983	2.45×10^{-4}	2.67×10^{-4}	2.65×10^{-4}	0.0116	0.0163
9	962	8.01×10^{-5}	8.30×10^{-5}	9.95×10^{-5}	0.0073	0.0100
10	367	1.42×10^{-4}	1.28×10^{-4}	1.40×10^{-4}	0.0095	0.0118



Hình 7. *MSE, MAE và RMSE trên test theo số nơ-ron B.*



Hình 8. *Train MSE và Validation MSE theo số nơ-ron B.*



Hình 9. *Learning curve (train, validation, epoch tốt nhất) của các mô hình B = 1 ... 10.*

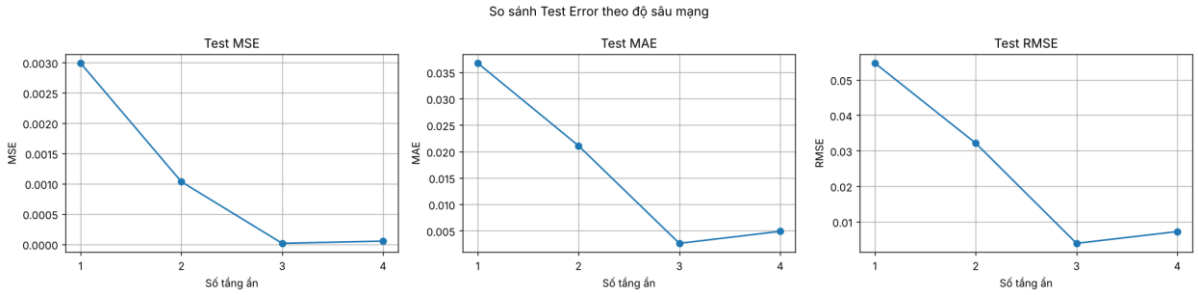
B = 1 cho kết quả kém nhất (MSE test 4.25×10^{-3}). Khi tăng B, sai số nhìn chung giảm và tốt nhất là B = 9 với 9.95×10^{-5}. Train MSE, Validation MSE và Test MSE cùng bậc và chênh lệch nhỏ ở mọi B, nên chưa thấy dấu hiệu overfitting. Hình 9 cho thấy train và validation giảm nhanh trong vài chục epoch đầu rồi giảm chậm dần; với phần lớn cấu hình, epoch tốt nhất nằm gần 1000 (Bảng 4).

4.5. Thí nghiệm C: số tầng ẩn

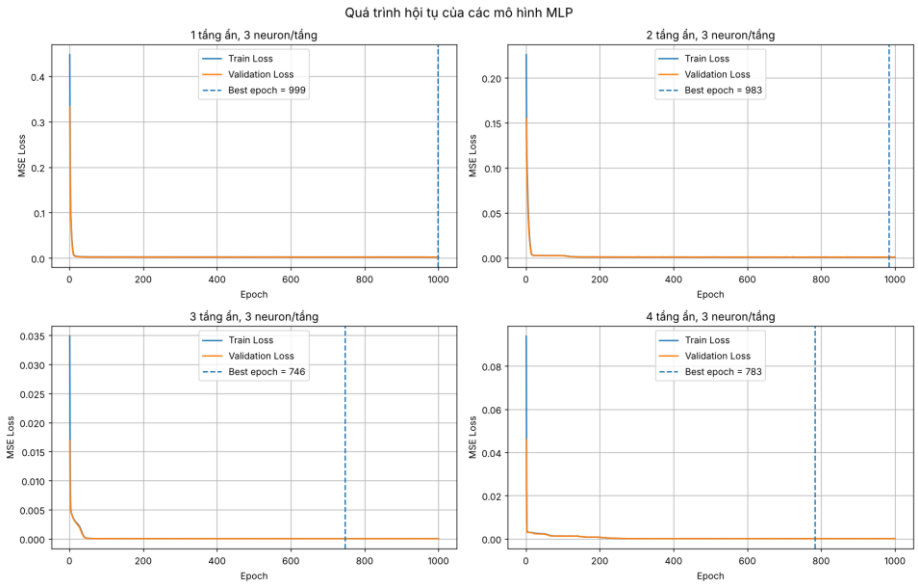
Với 1000 mẫu, số tầng ẩn C tăng từ 1 đến 4, mỗi tầng 3 nơ-ron (Bảng 5, Hình 10, 11, 12).

Bảng 5. *Kết quả thí nghiệm C (mỗi tầng ẩn 3 nơ-ron, n = 1000).*

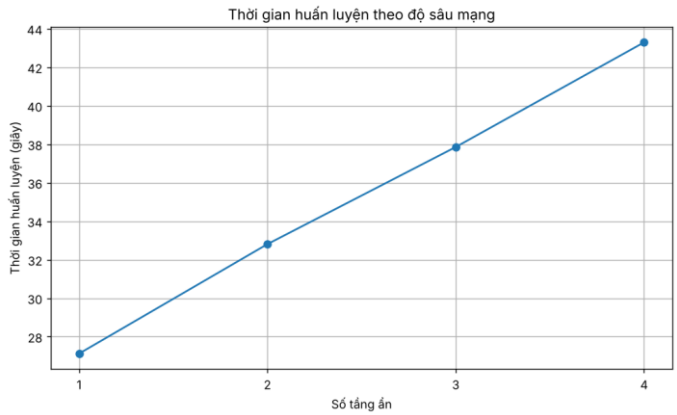
Số tầng ẩn	Epoch tốt nhất	Train MSE	Val MSE	Test MSE	Test MAE	Test RMSE	Thời gian (s)
1	999	2.18×10^{-3}	2.14×10^{-3}	2.99×10^{-3}	0.0367	0.0547	27.12
2	983	7.56×10^{-4}	7.86×10^{-4}	1.03×10^{-3}	0.0211	0.0321	32.81
3	746	1.23×10^{-5}	1.22×10^{-5}	1.52×10^{-5}	0.0026	0.0039	37.85
4	783	4.85×10^{-5}	5.04×10^{-5}	5.20×10^{-5}	0.0049	0.0072	43.29



Hình 10. *MSE, MAE và RMSE trên test theo số tầng ẩn.*



Hình 11. *Learning curve của các mô hình 1 … 4 tầng ẩn.*



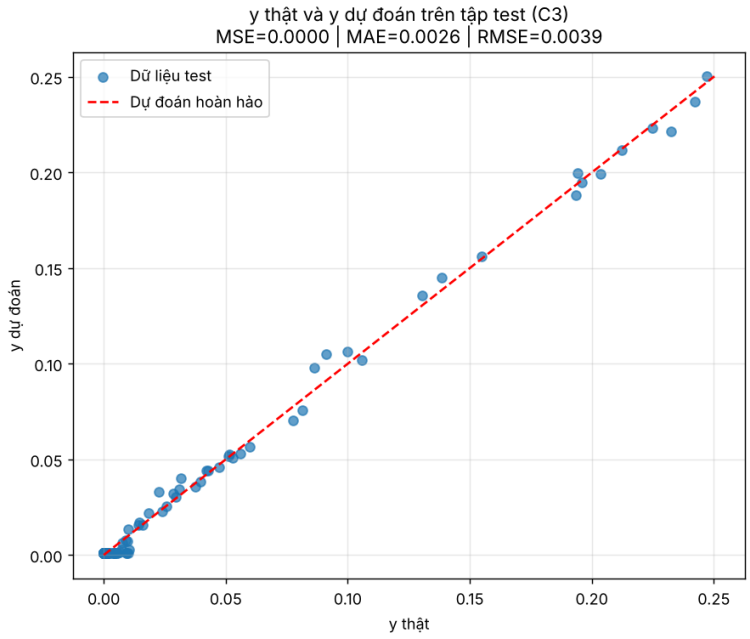
Hình 12. *Thời gian huấn luyện theo số tầng ẩn.*

Sai số giảm rất nhanh khi thêm tầng ẩn: MSE test là 2.99×10^{-3} (1 tầng), 1.03×10^{-3} (2 tầng) và 1.52×10^{-5} (3 tầng), thấp hơn 1 tầng khoảng 197 lần. Mạng 4 tầng cho 5.20×10^{-5}, kém hơn mạng 3 tầng. Thời gian huấn luyện tăng theo độ sâu, từ 27.12 s lên 43.29 s (Hình 12).

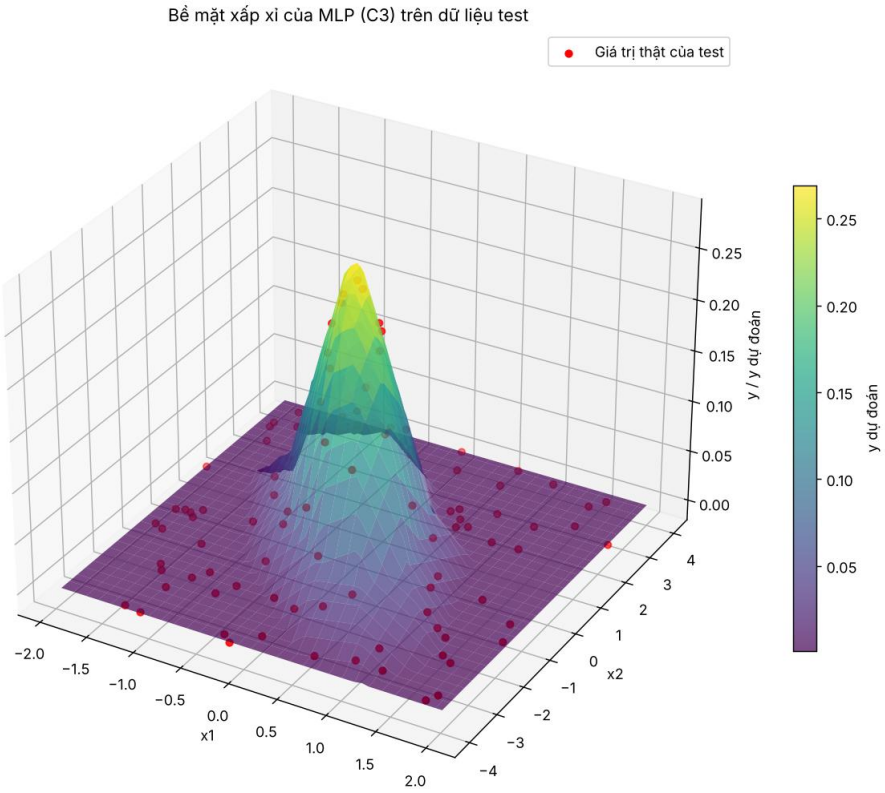
4.6. Kiến trúc được chọn và minh họa

Trong tất cả cấu hình của ba thí nghiệm A, B, C, mô hình có validation MSE nhỏ nhất là **3 tầng ẩn, mỗi tầng 3 nơ-ron (2-3-3-3-1)**, với Val MSE = 1.22×10^{-5}. Trên tập test (100 mẫu), mô hình đạt MSE = 1.52×10^{-5}, RMSE = 0.0039 và MAE = 0.0026. Trọng số được lưu tại weights/selected_model.pt. Hình 13 so sánh y dự đoán với y thật trên tập test, và Hình 14 là bề mặt xấp xỉ của mô hình cùng các điểm test.

Mô hình được kiểm tra lại bằng src/select_model.py: nạp weights/selected_model.pt cho đúng Val MSE và Test MSE đã lưu; tập train, validation và test (720/180/100) không giao nhau; cấu trúc lớp và số tham số (37) khớp với cấu hình. Kết quả kiểm tra ghi trong results/selection_check.json.



Hình 13. *y thật và y dự đoán trên tập test của kiến trúc được chọn.*



Hình 14. *Bề mặt xấp xỉ của kiến trúc được chọn và các điểm test.*

4.7. Thử nghiệm trên 3 seed

Để kiểm tra kết quả có phụ thuộc vào khởi tạo ngẫu nhiên hay không, mô hình chuẩn và mọi cấu hình của A, B, C được huấn luyện lại với 3 seed 42, 123 và 2024, các thiết lập khác giữ nguyên (Bảng 2). Chỉ thay đổi seed ở quyết định khởi tạo trọng số và thứ tự shuffle của DataLoader; dữ liệu và cách chia train/validation/test vẫn giữ cố định. Seed 42 là kết quả đã trình bày ở các mục 4.2 đến 4.6. Trọng số và log được lưu riêng theo thí nghiệm và theo seed (weights/expA_weight/seed_<s>/, logs/expA_log/seed_<s>/ và tương tự cho base, B, C). Bảng 6 là cấu hình có validation MSE nhỏ nhất ở từng seed, Bảng 7 là test MSE của từng cấu hình và Hình 15 đến 17 vẽ các kết quả này.

Bảng 6. *Cấu hình có validation MSE nhỏ nhất ở từng seed (trong ngoặc là test MSE).*

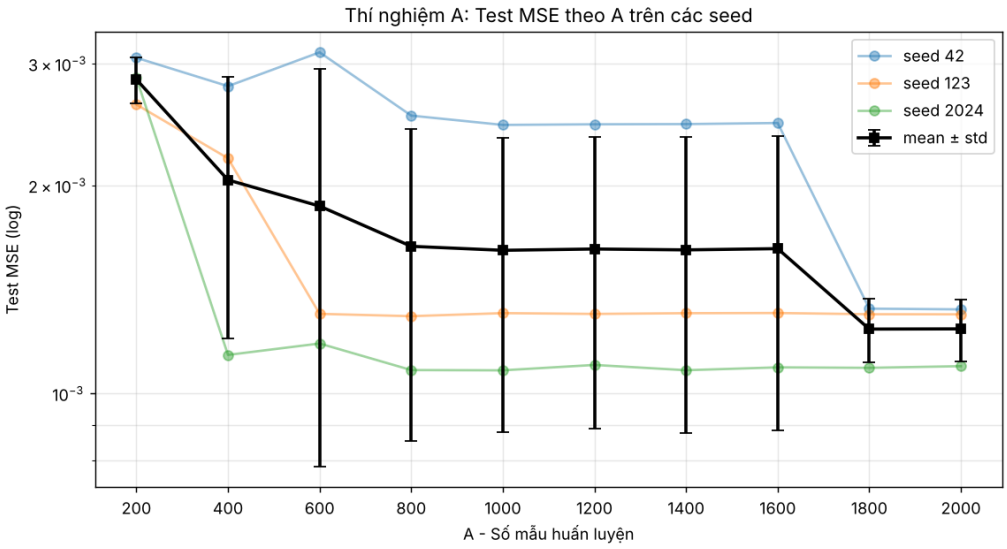
	Seed 42	**Seed 123**	**Seed 2024**
Thí nghiệm A	A1800 (test 1.33×10^{-3})	A1800 (test 1.30×10^{-3})	A1600 (test 1.09×10^{-3})
Thí nghiệm B	B9 (test 9.95×10^{-5})	B10 (test 1.98×10^{-4})	B10 (test 1.47×10^{-4})
Thí nghiệm C	C3 (test 1.52×10^{-5})	C4 (test 4.65×10^{-5})	C4 (test 4.80×10^{-5})

Bảng 7. *Test MSE của từng cấu hình trên 3 seed, trung bình và độ lệch chuẩn (mẫu).*

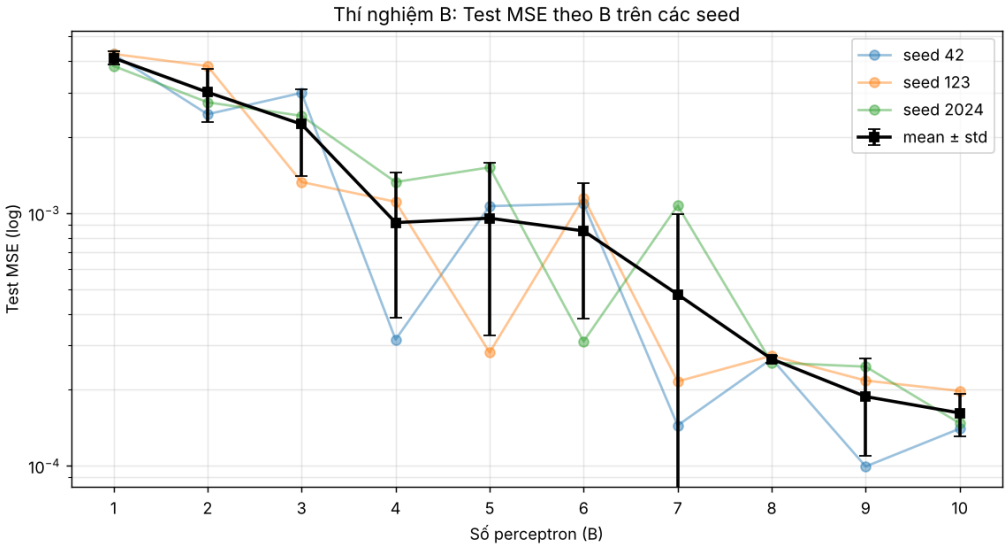
Cấu hình	**Test MSE, seed 42**	**Test MSE, seed 123**	**Test MSE, seed 2024**	**Trung bình ± độ lệch chuẩn**
base	3.03×10^{-3}	3.36×10^{-3}	3.37×10^{-3}	$3.25\times10^{-3} \pm 1.92\times10^{-4}$
A200	3.06×10^{-3}	2.62×10^{-3}	2.86×10^{-3}	$2.84\times10^{-3} \pm 2.21\times10^{-4}$

Cấu hình	Test MSE, seed 42	Test MSE, seed 123	Test MSE, seed 2024	Trung bình ± độ lệch chuẩn
A400	2.78×10^{-3}	2.19×10^{-3}	1.14×10^{-3}	$2.04\times10^{-3} \pm 8.33\times10^{-4}$
A600	3.11×10^{-3}	1.30×10^{-3}	1.18×10^{-3}	$1.87\times10^{-3} \pm 1.08\times10^{-3}$
A800	2.52×10^{-3}	1.29×10^{-3}	1.08×10^{-3}	$1.63\times10^{-3} \pm 7.78\times10^{-4}$
A1000	2.44×10^{-3}	1.31×10^{-3}	1.08×10^{-3}	$1.61\times10^{-3} \pm 7.31\times10^{-4}$
A1200	2.45×10^{-3}	1.30×10^{-3}	1.10×10^{-3}	$1.62\times10^{-3} \pm 7.28\times10^{-4}$
A1400	2.45×10^{-3}	1.31×10^{-3}	1.08×10^{-3}	$1.61\times10^{-3} \pm 7.35\times10^{-4}$
A1600	2.46×10^{-3}	1.31×10^{-3}	1.09×10^{-3}	$1.62\times10^{-3} \pm 7.36\times10^{-4}$
A1800	1.33×10^{-3}	1.30×10^{-3}	1.09×10^{-3}	$1.24\times10^{-3} \pm 1.30\times10^{-4}$
A2000	1.32×10^{-3}	1.30×10^{-3}	1.09×10^{-3}	$1.24\times10^{-3} \pm 1.26\times10^{-4}$
B1	4.25×10^{-3}	4.26×10^{-3}	3.82×10^{-3}	$4.11\times10^{-3} \pm 2.51\times10^{-4}$
B2	2.46×10^{-3}	3.82×10^{-3}	2.75×10^{-3}	$3.01\times10^{-3} \pm 7.16\times10^{-4}$
B3	2.99×10^{-3}	1.33×10^{-3}	2.43×10^{-3}	$2.25\times10^{-3} \pm 8.46\times10^{-4}$
B4	3.16×10^{-4}	1.11×10^{-3}	1.33×10^{-3}	$9.18\times10^{-4} \pm 5.33\times10^{-4}$
B5	1.07×10^{-3}	2.81×10^{-4}	1.52×10^{-3}	$9.55\times10^{-4} \pm 6.26\times10^{-4}$
B6	1.09×10^{-3}	1.15×10^{-3}	3.11×10^{-4}	$8.51\times10^{-4} \pm 4.68\times10^{-4}$
B7	1.45×10^{-4}	2.16×10^{-4}	1.07×10^{-3}	$4.77\times10^{-4} \pm 5.16\times10^{-4}$
B8	2.65×10^{-4}	2.73×10^{-4}	2.56×10^{-4}	$2.65\times10^{-4} \pm 8.25\times10^{-6}$
B9	9.95×10^{-5}	2.18×10^{-4}	2.47×10^{-4}	$1.88\times10^{-4} \pm 7.81\times10^{-5}$
B10	1.40×10^{-4}	1.98×10^{-4}	1.47×10^{-4}	$1.62\times10^{-4} \pm 3.14\times10^{-5}$
C1	2.99×10^{-3}	1.33×10^{-3}	2.43×10^{-3}	$2.25\times10^{-3} \pm 8.46\times10^{-4}$
C2	1.03×10^{-3}	1.66×10^{-4}	1.78×10^{-4}	$4.59\times10^{-4} \pm 4.97\times10^{-4}$
C3	1.52×10^{-5}	2.17×10^{-3}	1.64×10^{-3}	$1.28\times10^{-3} \pm 1.12\times10^{-3}$
C4	5.20×10^{-5}	4.65×10^{-5}	4.80×10^{-5}	$4.88\times10^{-5} \pm 2.83\times10^{-6}$

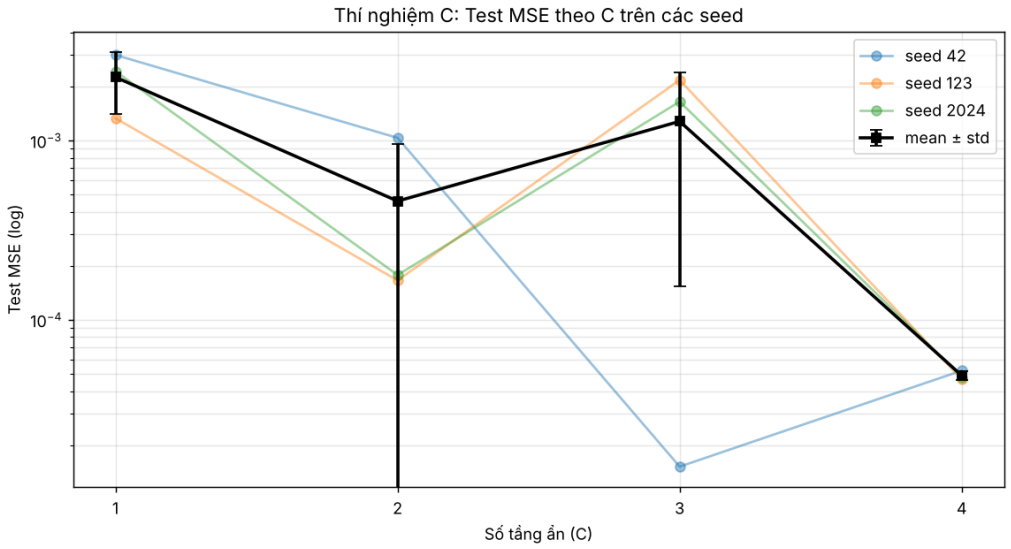
Nhận xét. Mô hình chuẩn khá ổn định giữa các seed (test MSE $3.25\times10^{-3} \pm 1.92\times10^{-4}$). Ở thí nghiệm B, xu hướng giảm sai số khi tăng số nơ-ron giữ nguyên ở cả 3 seed: test MSE trung bình giảm từ 4.11×10^{-3} (B = 1) xuống 1.62×10^{-4} (B = 10). Ở thí nghiệm A (validation và test cố định), test MSE trung bình giảm từ 2.84×10^{-3} (A = 200) xuống 1.24×10^{-3} (A = 2000); đường cong gần như phẳng quanh 1.6×10^{-3} từ A = 800 đến A = 1600 rồi giảm ở A = 1800 và 2000, nơi độ lệch chuẩn cũng nhỏ nhất (1.30×10^{-4} ở A = 1800). Cấu hình tốt nhất theo validation MSE thay đổi theo seed (Bảng 6).



Hình 15. *Thí nghiệm A: test MSE theo số mẫu train trên 3 seed (đường mờ là từng seed, đường đen là trung bình ± độ lệch chuẩn; trục y theo log).*



Hình 16. *Thí nghiệm B: test MSE theo số nơ-ron trên 3 seed.*



Hình 17. *Thí nghiệm C: test MSE theo số tầng ẩn trên 3 seed.*

Thí nghiệm C. Mô hình 4 tầng ẩn cho kết quả ổn định nhất: test MSE $4.88\times10^{-5} \pm 2.83\times10^{-6}$ và được chọn ở seed 123 và 2024. Mô hình 3 tầng ẩn, được chọn ở mục 4.6 với seed 42 (test MSE 1.52×10^{-5}), chỉ đạt 2.17×10^{-3} ở seed 123 và 1.64×10^{-3} ở seed 2024, trung bình $1.28\times10^{-3} \pm 1.12\times10^{-3}$. Như vậy kết quả rất tốt của 3 tầng ẩn ở seed 42 phần lớn do khởi tạo thuận lợi; kết luận ở mục 4.6 (kiến trúc 2-3-3-3-1) đúng cho seed 42, còn khi xét 3 seed thì 4 tầng ẩn là cấu hình ổn định hơn.

Lưu ý: chỉ có 3 seed nên độ lệch chuẩn chỉ mang tính tham khảo.

5. Thảo luận

5.1. Diễn giải xu hướng

- **Số mẫu (A):** thêm dữ liệu giúp giảm sai số, nhưng mức giảm khiêm tốn (MSE vẫn ở cỡ 10^{-3}) so với việc đổi kiến trúc ở B và C. Với mạng chỉ gồm 1 tầng ẩn × 3 nơ-ron, sai số có vẻ bị giới hạn bởi cấu trúc mạng hơn là bởi lượng dữ liệu.
- **Số nơ-ron (B):** nhiều nơ-ron hơn cho sai số thấp hơn.
- **Số tầng ẩn (C):** thêm tầng ẩn giảm sai số mạnh nhất.

5.2. Các trường hợp không như kỳ vọng

- **A = 600 tệ hơn A = 400, và đường cong A gần như phẳng từ A = 800 đến A = 1600 rồi giảm ở A = 1800.** Validation và test của A cố định nên khác biệt này không đến từ tập test; có thể do mạng nhỏ (1 tầng ẩn × 3 nơ-ron) rơi vào các nghiệm khác nhau và do epoch tốt nhất của A nằm trong khoảng 732 đến 985 trên 1000 epoch, tức là chưa chắc đã hội tụ hết. Ở mục 4.7, đường trung bình 3 seed cũng có dạng bậc thang: phẳng quanh 1.6×10^{-3} rồi giảm ở A = 1600 đến 1800 tùy seed.
- **B = 3 kém hơn B = 2, B = 4 tốt hơn nhiều so với B = 5 và 6, B = 10 kém hơn B = 9.** Kết quả thay đổi theo seed (mục 4.7) và phần lớn có epoch tốt nhất gần 1000 (Bảng 4),

tức là chưa chắc đã hội tụ. Vì vậy chênh lệch giữa các B lân cận có thể đến từ lần khởi tạo và độ dài huấn luyện chứ không chỉ từ số nơ-ron.

5.3. Hạn chế

- Mỗi cấu hình mới chạy 3 seed (mục 4.7); với số seed ít, trung bình và độ lệch chuẩn chỉ mang tính tham khảo, khó kết luận chắc chắn với các chênh lệch nhỏ. Seed chỉ đổi khởi tạo và thứ tự shuffle, dữ liệu và cách chia tập vẫn cố định.
- Nhiều mô hình có epoch tốt nhất gần 1000 (Bảng 4, 5), nên có thể chưa hội tụ hoàn toàn.
- Ở thí nghiệm A, các tập train lồng nhau (luôn lấy n mẫu đầu của phần còn lại) nên các điểm A không độc lập hoàn toàn với nhau.
- Đầu vào và đầu ra không được chuẩn hóa.
- Thời gian huấn luyện chỉ đo một lần trên CPU, có thể chịu ảnh hưởng của tải máy.

5.4. Đề xuất cải tiến

- Chạy mỗi cấu hình với nhiều seed hơn (hiện là 3) và cho seed thay đổi cả dữ liệu và cách chia tập.
- Thử mạng lớn hơn cho thí nghiệm A, vì với 1 tầng ẩn × 3 nơ-ron sai số có vẻ bị giới hạn bởi cấu trúc mạng hơn là bởi lượng dữ liệu.
- Dùng bộ lập lịch learning rate hoặc huấn luyện lâu hơn để các mô hình hội tụ đầy đủ.
- Chuẩn hóa đầu vào và đầu ra.

6. Kết luận

Báo cáo đã cài đặt MLP bằng PyTorch để xấp xỉ hàm Gaussian 2D và khảo sát ba yếu tố. Kết quả chính: (1) tăng số mẫu từ 200 lên 2000 với mạng 1 tầng ẩn × 3 nơ-ron giảm MSE test từ 3.06×10^{-3} xuống 1.32×10^{-3}; (2) tăng số nơ-ron của một tầng ẩn giảm sai số rõ rệt, tốt nhất ở B = 9 (9.95×10^{-5}); (3) tăng số tầng ẩn hiệu quả nhất: mạng 3 tầng × 3 nơ-ron đạt MSE test 1.52×10^{-5}. Khi chạy thêm seed 123 và 2024 (mục 4.7), kết quả rất tốt của mạng 3 tầng ở seed 42 không lặp lại (mạng 4 tầng ổn định hơn), còn xu hướng giảm sai số khi tăng số mẫu và số nơ-ron được giữ nguyên. Các điểm không đơn điệu giữa những cấu hình lân cận có thể do khởi tạo ngẫu nhiên (mục 4.7) và do quá trình huấn luyện. Hướng tiếp theo là chạy thêm nhiều seed, thử mạng lớn hơn cho A và huấn luyện đầy đủ hơn.

Tài liệu tham khảo

[1] I. Goodfellow, Y. Bengio, A. Courville. Deep Learning. MIT Press, 2016.

[2] G. Cybenko. Approximation by superpositions of a sigmoidal function. Mathematics of Control, Signals and Systems, 2(4), 303–314, 1989.

[3] K. Hornik, M. Stinchcombe, H. White. Multilayer feedforward networks are universal approximators. Neural Networks, 2(5), 359–366, 1989.

[4] D. P. Kingma, J. Ba. Adam: A method for stochastic optimization. ICLR, 2015.

[5] A. Paszke et al. PyTorch: An imperative style, high-performance deep learning library. NeurIPS, 2019.